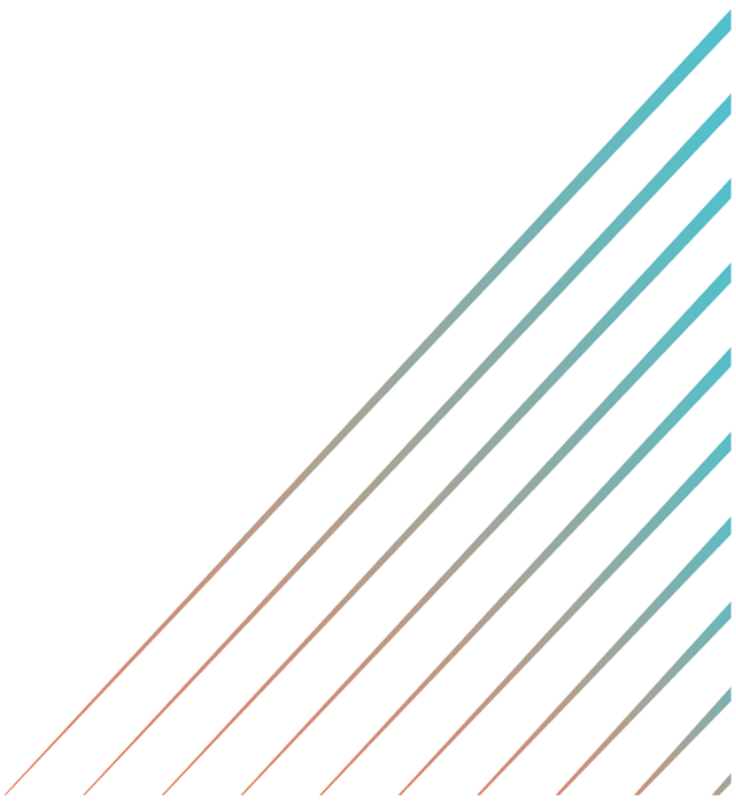


PORTADA



Documentación

BreathTakingTravels



Contenido

1. Descripción general 3

3. Diagrama de entidad-relación(E-R) 5

4. Identificación de requisitos 6

5. Actores del sistema 8

5.1. Descripción de los Casos de Uso 10

6. Diseño — Arquitectura del sistema 17

6.1. Diseño del sistema 17

6.2. APIs externas integradas 18

7. Implementación 20

7.1. Tecnologías utilizadas 20

7.2. Estructura del proyecto 22

8. Validación y pruebas 23

9. Análisis de costes 25

 10. Financiación 26

11. Plan de Producción y Recursos Humanos(RRHH) 27

12. Plan de prevención de riesgos laborales 29

Tablas

Tabla 1. Requisitos funcionales, de integración/implementación y no funcionales 6

Tabla 2. Secuencia de tablas de cada caso de uso por separado 10

Tabla 3. APIs externas concretas y su uso dentro de BTT 18

Tabla 4. Tabla que explica cada tecnología, versión y utilización en el frontend 20

Tabla 5. Tabla que explica cada tecnología, versión y utilización en el backend 21

Tabla 6. Tabla que explica cada tecnología, versión y utilización en la base de datos 21

Tabla 7. Casos de prueba, resultados y observaciones de BTT 23

Tabla 8. Roles que se han necesitado para crear BTT 27

Tabla 9. Tabla que muestra las fases, descripción y tiempo de cada fase 28

Tabla 10. Riesgos laborales y las medidas para prevenirlas 29

Ilustraciones

Ilustración 1. Modelo de Tablas de BTT 4

Ilustración 2. Modelo Entidad-Relación de BTT..... 5

Ilustración 3. Representación de un diagrama de casos de uso 9

Ilustración 4. Arquitectura del sistema de BTT 19

1. Descripción general

BreathTakingTravels(BTT) es una plataforma web de planificación de viajes multidesino que permite al usuario diseñar su ruta personalizada desde cero. El usuario elige su país de origen y selecciona múltiples destinos — con un mínimo de dos — pudiendo consultar vuelos, hoteles y actividades recomendadas según el clima y sus preferencias para cada destino.

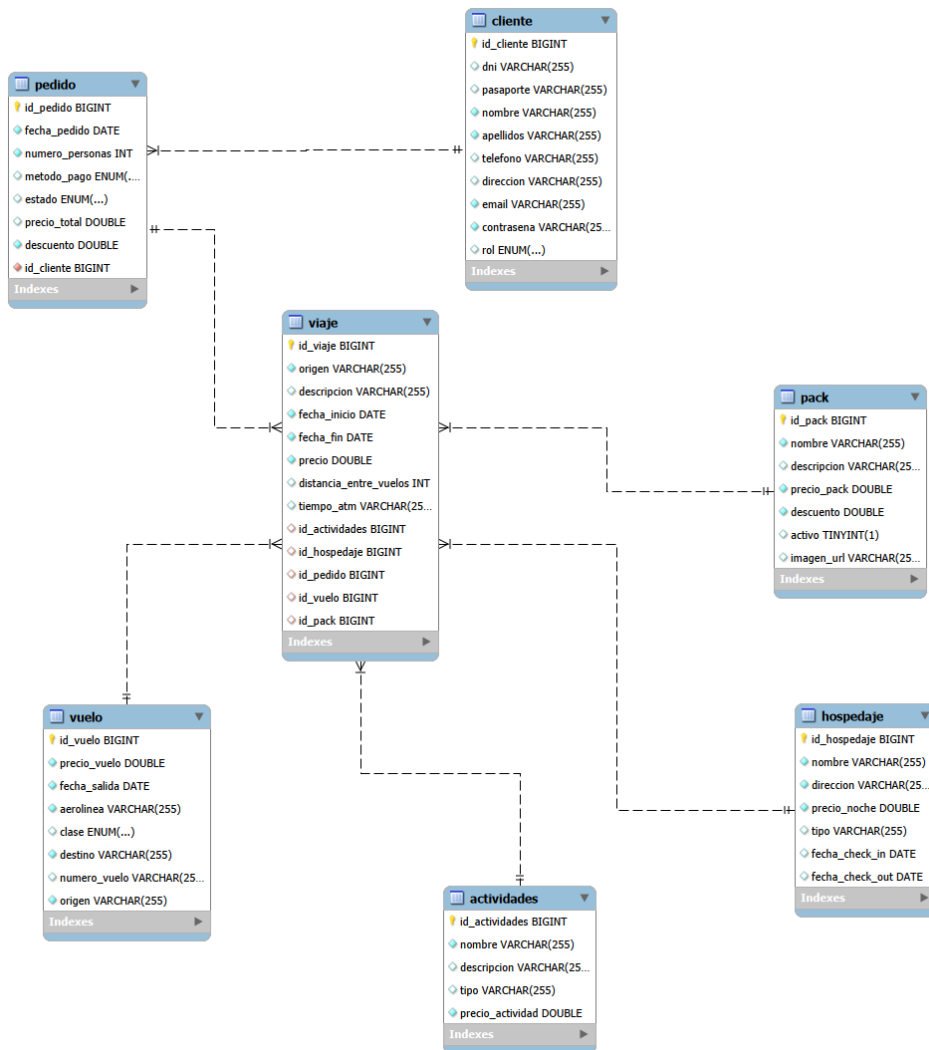
La plataforma prioriza los destinos más cercanos al origen para que los vuelos sean más económicos. Una vez completado el viaje, el usuario confirma el pago y el pedido queda registrado en su historial personal.

Además, dispone de paquetes turísticos predefinidos con descuentos que pueden reservarse directamente, y de un panel de administración completo para gestionar pedidos, clientes y packs.

La plataforma está destinada a usuarios que deseen planificar un viaje multidesino de forma autónoma, sin necesidad de consultar múltiples plataformas ni depender de agencias de viajes, priorizando siempre las opciones más económicas

2. Modelo de tablas

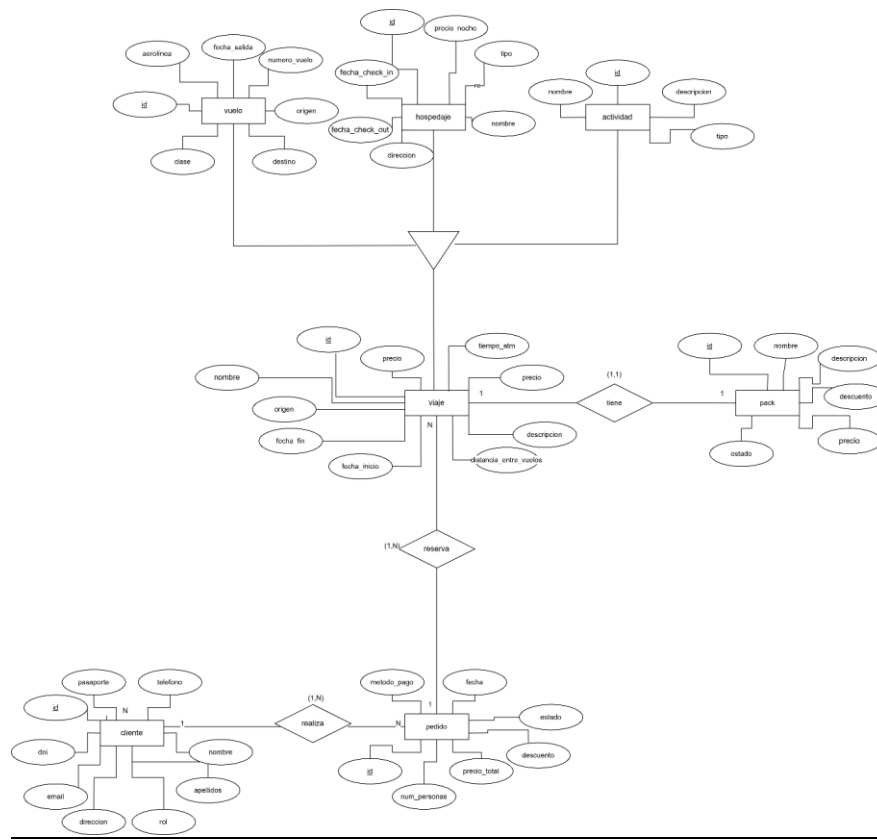
Ilustración 1. Modelo de Tablas de BTT



Nota: Hecho con MySQL Workbench con la herramienta de Reverse Engineer

3. Diagrama de entidad-relación(E-R)

Ilustración 2. Modelo Entidad-Relación de BTT



Nota: Está realizado con draw.io y se ha sometido a múltiples cambios durante el desarrollo del proyecto.

4. Identificación de requisitos

Tabla 1. Requisitos funcionales, de integración/implementación y no funcionales

Código	Tipo	Descripción
RF1	Funcionalidad	Se pueden buscar y comparar múltiples destinos al mismo tiempo.
RF2	Funcionalidad	Se sugieren los destinos más económicos en base a vuelo+hospedaje+actividades+clima+distancia entre sitio a y sitio b.
RF3	Funcionalidad	Se pueden filtrar resultados por precio, valoración y relación calidad-precio en hoteles
RF4	Funcionalidad	Se muestran los precios actualizados de vuelos y hospedaje mediante APIs externas.
RF5	Funcionalidad	Se pueden ver una lista de actividades y planes recomendados para cada destino.
RF6	Funcionalidad	Se gestionan cuentas de usuario con inicio de sesión y registro.
RF7	Funcionalidad	El usuario podrá registrarse en la página con el email/teléfono y una contraseña propia.
RF8	Funcionalidad	Una vez registrado, el usuario podrá iniciar sesión solo con poner su email y su contraseña.
RF9	Funcionalidad	El pago se procesa mediante PayPal sandbox con validación de datos una vez termina de crear su viaje.
RF10	Funcionalidad	Se cuenta con un sistema de pago que aceptaría métodos de pago modernos como tarjetas de crédito, PayPal.
RF11	Funcionalidad	Panel de administración para gestionar pedidos, clientes, viajes y packs
RF12	Funcionalidad	Actividades filtradas por categoría y clima
RF13	Funcionalidad	Paquetes turísticos predefinidos con reserva directa
RI1	Interfaz Usuario	Se pueden seleccionar múltiples destinos en un mismo formulario.
RI2	Interfaz Usuario	Se muestra un resumen comparativo visual con precios, distancias, fechas, etc.
RI3	Interfaz Usuario	Resumen de ruta en tiempo real con precio actualizado
RI4	Interfaz Usuario	Se pueden acceder a los filtros cuando creas un viaje y buscas hospedaje
RI5	Software	Se conecta con APIs de vuelos.
RI6	Software	Se conecta con APIs de hospedaje.

R17	Software	Se conecta con APIs de clima
R18	Software	Se conecta con APIs de localización
R19	Software	Se conecta con APIs de países
R110	Software	Se conecta con APIs de pagos
R111	Comunicaciones	Se incluyen formularios de contacto
RNF1	Usabilidad	La interfaz es simple, intuitiva y debe estar optimizada para móviles.
RNF2	Accesibilidad	Se cuenta texto alternativo en todas las imágenes.
RNF3	Accesibilidad	Se puede navegar con teclado y ratón.
RNF4	Seguridad	La información del usuario estará cifrada y protegida.
RNF5	Rendimiento	Las comparativas deben generarse en un tiempo óptimo.
RNF6	Compatibilidad	Compatible con los navegadores modernos que usen Google como motor de búsqueda.
RNF7	Fiabilidad	Funcionará de manera correcta el 99% de las ocasiones
RNF8	Mantenibilidad	El código estará documentado y versionado para facilitar actualizaciones.
RNF9	Desempeño	Optimizado para motores de búsqueda para que cargue en un tiempo no mayor a 5 segundos.

Nota: Todos estos han sido probados múltiples veces antes de añadirlos a la lista. Durante el

transcurso del proyecto, se han modificado múltiples veces.

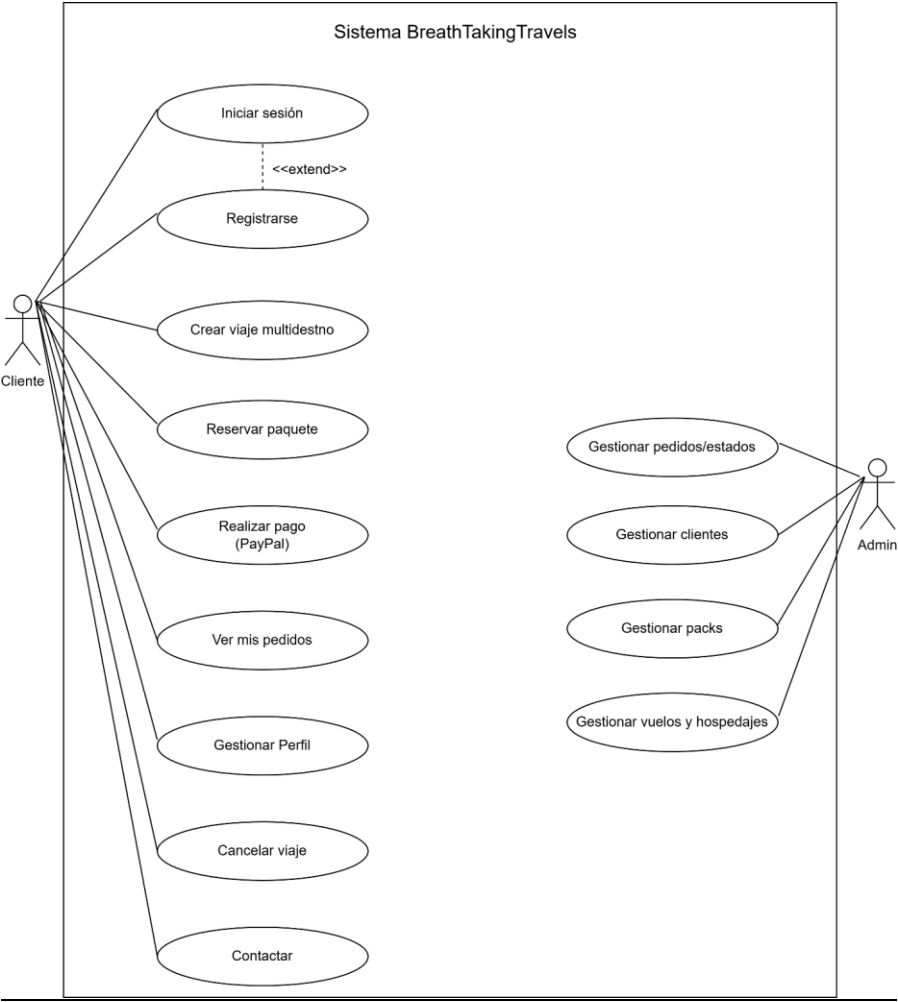
5. Actores del sistema

El sistema cuenta con dos actores principales:

Cliente: Usuario registrado en la plataforma. Puede crear viajes multidesino personalizados, reservar paquetes turísticos, realizar pagos mediante PayPal, consultar sus pedidos y gestionar su perfil. Para acceder a las funcionalidades principales debe estar autenticado.

Administrador: Usuario con privilegios elevados que gestiona el sistema a través del panel de administración. Puede gestionar pedidos y cambiar sus estados, administrar clientes y sus roles, crear y editar paquetes turísticos, y consultar vuelos y hospedajes registrados en el sistema.

Ilustración 3. Representación de un diagrama de casos de uso



Nota: Este diagrama ha sido realizado con draw.io.

5.1. Descripción de los Casos de Uso

Tabla 2. Secuencia de tablas de cada caso de uso por separado

Caso de uso	CU01 — Iniciar sesión
Actor	Cliente
Precondición	El usuario debe estar registrado en la página
Flujo normal	El usuario introduce email y contraseña y accede al sistema
Flujo alternativo	Las credenciales son incorrectas(contraseña incorrecta o el email no está registrado) — se muestra mensaje de error

Caso de uso	CU02 — Registrarse
Actor	Cliente
Precondición	El email no debe estar registrado previamente
Flujo normal	El usuario rellena el formulario con sus datos y se crea la cuenta
Flujo alternativo	El email ya existe — se muestra mensaje de error

Caso de uso	CU03 — Crear viaje multidesino
Actor	Cliente
Precondición	El usuario debe estar autenticado
Flujo normal	El usuario elige origen, destinos, vuelos, hoteles y actividades. Añade mínimo 2 destinos(países distintos) y un vuelo de vuelta(en el mismo sitio del origen), y confirma el viaje mediante PayPal
Flujo alternativo	No hay vuelos disponibles para la ruta o fecha seleccionada — se muestra mensaje de error

Caso de uso	CU04 — Reservar paquete
Actor	Cliente
Precondición	El usuario debe estar autenticado y existir paquetes activos
Flujo normal	El usuario selecciona un pack, elige fecha de inicio, número de personas y método de pago, y confirma la reserva
Flujo alternativo	No hay paquetes disponibles — se muestra mensaje informativo

Caso de uso	CU05 — Realizar pago (PayPal)
Actor	Cliente
Precondición	El usuario debe tener un viaje o paquete listo para confirmar
Flujo normal	Se muestra el resumen del pago, el usuario completa el pago mediante PayPal sandbox y se crea el pedido
Flujo alternativo	El pago es cancelado o falla — el pedido no se crea y se muestra mensaje de error

Caso de uso	CU06 — Ver mis pedidos
Actor	Cliente
Precondición	El usuario debe estar autenticado y tener pedidos realizados
Flujo normal	El usuario accede a su historial de pedidos y puede ver el detalle de cada viaje
Flujo alternativo	No hay pedidos — se muestra mensaje informativo

Caso de uso	CU07 — Gestionar perfil
Actor	Cliente
Precondición	El usuario debe estar autenticado
Flujo normal	El usuario consulta sus datos personales y puede cerrar sesión
Flujo alternativo	—

Caso de uso	CU08 — Cancelar viaje
Actor	Cliente
Precondición	El usuario debe tener pedidos en estado PENDIENTE
Flujo normal	El administrador cambia el estado del pedido a CANCELADO desde el panel
Flujo alternativo	—

Caso de uso	CU09 — Contactar
Actor	Cliente
Precondición	Ninguna
Flujo normal	El usuario accede al formulario de contacto y envía un mensaje
Flujo alternativo	—

Caso de uso	CU10 — Gestionar pedidos/estados
Actor	Administrador
Precondición	El usuario debe estar autenticado como administrador
Flujo normal	El administrador lista todos los pedidos, cambia su estado y puede eliminarlos
Flujo alternativo	—

Caso de uso	CU11 — Gestionar clientes
Actor	Administrador
Precondición	El usuario debe estar autenticado como administrador
Flujo normal	El administrador lista todos los clientes, cambia su rol y puede eliminarlos
Flujo alternativo	—

Caso de uso	CU12 — Gestionar packs
Actor	Administrador
Precondición	El usuario debe estar autenticado como administrador
Flujo normal	El administrador crea, edita y elimina paquetes turísticos con imagen y precio
Flujo alternativo	—

Caso de uso	CU13 — Gestionar vuelos y hospedajes
Actor	Administrador
Precondición	El usuario debe estar autenticado como administrador
Flujo normal	El administrador consulta los vuelos y hospedajes registrados y puede eliminarlos
Flujo alternativo	—

Nota: Cada uno de los casos de uso posible junto a las condiciones y el flujo que sería el normal, que sucede más veces y el alternativo, menos común.

6. Diseño — Arquitectura del sistema

Este apartado explica cómo está estructurado el sistema. Te propongo este texto para la memoria:

6.1. Diseño del sistema

Arquitectura

La aplicación sigue una arquitectura cliente-servidor de tres capas:

- **Capa de presentación** : Frontend desarrollado en React 19 con Vite y como entorno de ejecución e instalación paquetes npm, *Node.js*. Se comunica con el backend mediante peticiones HTTP REST y con APIs externas directamente desde el navegador.
- **Capa de negocio** : Backend desarrollado en Spring Boot 3.3.5 con Java 17. Expone una API REST consumida por el frontend, gestiona la lógica de negocio, la autenticación mediante JWT(JSON Web Token) y actúa como proxy para la API de Duffel.
- **Capa de datos** : Base de datos relacional MySQL gestionada mediante JPA/Hibernate con Spring Data. Se hizo uso muchas veces de Postman para testear el Backend.

6.2. APIs externas integradas

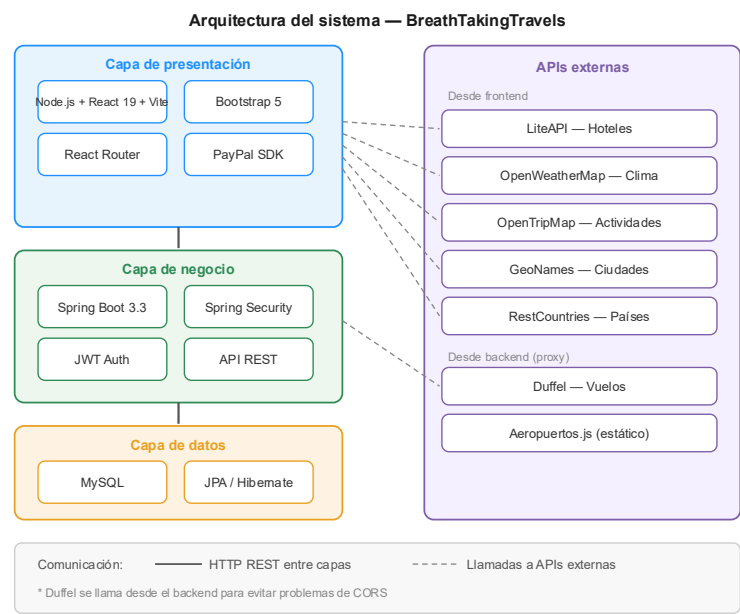
Tabla 3. APIs externas concretas y su uso dentro de BTT

API externa	Uso
Duffel	Búsqueda de vuelos y aeropuertos(código) en tiempo real
LiteAPI	Búsqueda de hoteles con precios y valoración actualizados
OpenWeatherMap	Clima actual de la ciudad de destino para filtrar actividades
OpenTripMap	Puntos de interés y actividades por ciudad
GeoNames	Ciudades principales por país
RestCountries	Países con banderas, coordenadas y área
PayPal Sandbox	Procesamiento de pagos en modo prueba

Comentado [MG1]: Dado que la API de Duffel no devuelve aeropuertos de la forma que se estimaba, se implementó una lista estática de aeropuertos (carpeta data) principales por país en el frontend (aeropuertos.js) como fallback cuando la API no dispone de datos para un país concreto . Aunque siempre se tire de esta API porque Duffel no funciona como esperábamos.

Nota: Esta tabla contiene todas las utilidades de cada API y sus nombre concretos. Para hacer uso de la mayoría debes registrarte en dicha página y vincularlo con la API key que generes.

Ilustración 4. Arquitectura del sistema de BTT



Nota: No fue planeada esta estructura de primera mano, de hecho un dato curioso es que si accedemos al documento Backend de Springboot el nombre de Angular está por todos lados.

7. Implementación

7.1. Tecnologías utilizadas

Frontend

Tabla 4. Tabla que explica cada tecnología, versión y utilización en el frontend

Tecnología	Versión	Uso
Node.js	20+	Entorno de ejecución
React (VS code)	19	Librería de interfaz de usuario
Vite	6	Empaquetador y servidor de desarrollo
Bootstrap	5	Framework de estilos CSS
Bootstrap Icons	1.11	Iconografía
React Router DOM	6	Navegación entre páginas
Axios	1.7	Peticiones HTTP al backend
Paypal sandbox	8	Integración de pagos PayPal

Backend

Tabla 5. Tabla que explica cada tecnología, versión y utilización en el backend

Tecnología	Versión	Uso
Java	17	Lenguaje de programación
Spring Boot (IntelliJ Community Edition)	3.3.5	Framework principal del backend
Spring Security	6	Autenticación y autorización
Jwt	0.11.5	Generación y validación de tokens JWT
Spring Data JPA	3.3.5	Acceso a la base de datos
Hibernate	6.5	ORM para mapeo objeto-relacional
MySQL Connector	8.3	Conexión con la base de datos
Lombok	1.18	Reducción de código boilerplate
SpringDoc OpenAPI	2.5	Documentación automática con Swagger

Base de datos

Tabla 6. Tabla que explica cada tecnología, versión y utilización en la base de datos

Tecnología	Versión	Uso
MySQL (MySQL Workbench)	8	Base de datos relacional

Nota: Todos y cada uno de los programas, APIs, aplicaciones, plataformas son todos los que han sido necesarios para crear la implementación de BTT. Vuelvo a destacar que para algunos programas. Entre paréntesis los entornos de desarrollo integrado.

7.2. Estructura del proyecto

Frontend (src/)

pages/ → Páginas principales (Home, CrearViaje, Admin, MisPedidos...)(.jsx, .css)

components/(crearViaje) → Componentes reutilizables (Navbar, ModalPago, FiltrosHoteles...)(.js,.css)

services/ → Llamadas a APIs (duffelService, hotelService, adminService...)(.js)

context/ → Contexto de autenticación (AuthContext)(.jsx para controlar el token del login y el registro)

data/ → Datos estáticos (aeropuertos.js)

Backend (src/main/java/)

controller/ → Controladores REST (ClienteController, PedidoController...)

service/ → Lógica de negocio (PedidoService, ViajeService...)

repository/ → Repositorios JPA (ClienteRepository, PedidoRepository...)

model/ → Entidades JPA (Cliente, Pedido, Viaje...)

dto/ → Objetos de transferencia de datos

security/ → Configuración JWT y Spring Security

enums/ → Enumeraciones (RolEnum, EstadoPedidoEnum...)

8. Validación y pruebas

Tabla 7. Casos de prueba, resultados y observaciones de BTT

ID	Caso de prueba	Resultado esperado	Resultado obtenido	Observaciones
P01	Registro de nuevo usuario	Se crea la cuenta y redirige al login	✓ Correcto	Puede quedarse sesión iniciada y si se reinicia, el token tiene fecha de expiración de 1 día.
P02	Login con credenciales correctas	Accede al sistema con token JWT	✓ Correcto	
P03	Login con credenciales incorrectas	Muestra mensaje de error	✓ Correcto	
P04	Crear viaje con 2 destinos	Se habilita el vuelo de vuelta	✓ Correcto	
P05	Buscar vuelos con Duffel	Se muestran hasta 5 vuelos	✓ Correcto	Puede tardar varios segundos (entre 2 y 5)
P06	Buscar hoteles con LiteAPI	Se muestran hoteles con precio	✓ Correcto	Algunos hoteles no tienen precio disponible, aunque esos suelen filtrarse
P07	Filtrar hoteles por precio	Se ocultan hoteles fuera del rango	✓ Correcto	
P08	Seleccionar actividades por categoría	Se filtran según clima y categoría	⚠ Parcial	Algunas categorías devuelven pocos resultados según la ciudad
P09	Confirmar pago con PayPal sandbox	Se crea el pedido	✓ Correcto	Solo funciona en modo sandbox
P10	Ver detalle de pedido	Se muestran los tramos del viaje	✓ Correcto	

ID	Caso de prueba	Resultado esperado	Resultado obtenido	Observaciones
P11	Reservar paquete turístico	Se crea el pedido con precio correcto	✓ Correcto	
P12	Acceso admin sin rol ADMIN	Página en blanco(blank) sin posibilidad de interactuar	✓ Correcto	Solo puedes pulsar en el logo para volver al inicio/funcionalidades del nav
P13	Cambiar estado de pedido	El estado se actualiza	✓ Correcto	
P14	Crear pack con imagen	El pack aparece en Home	✓ Correcto	Solo acepta URL de imagen
P15	Cambiar rol de cliente	Accede al panel admin	✓ Correcto	Requiere volver a iniciar sesión
P16	Buscar vuelos para país sin aeropuerto en Duffel	Se usan aeropuertos del fichero estático	✓ Correcto	Cobertura limitada a países principales

Nota: Están documentadas todas y cada una de las pruebas junto a unas observaciones para entender su utilidad sin siquiera tener que entrar.

9. Análisis de costes

Concepto	Coste estimado
Desarrollo (100h × 10,55€/h junior)	1.055 €
Herramientas de desarrollo (IntelliJ, VS Code)	0 €
APIs en fase de desarrollo (sandbox)	0 €
Hosting VPS producción (AWS)	10 €/mes
Base de datos cloud (AWS RDS)	5 €/mes
Dominio web	10 €/año
Total desarrollo	1.055 €
Total infraestructura anual	190 €/año

Comentado [MG2]: La media de lo que ganaría un programador junior según la media anual que es de 21.938€

10. Financiación

La plataforma BTT podría financiarse mediante los siguientes modelos:

- **Comisión por reserva** : un porcentaje del **2-5%** sobre cada viaje confirmado a través de la plataforma, cobrado o a las agencias o proveedores integrados.
 - **Plan premium** : suscripción mensual que desbloquea funcionalidades avanzadas como alertas de bajada de precios, viajes guardados o acceso prioritario a ofertas. En general, mayor interacción con el cliente que ha decidido confiar en nuestros servicios.
 - **Publicidad segmentada** : anuncios de aerolíneas, cadenas hoteleras y empresas de actividades turísticas dirigidos según el perfil y destinos del usuario.
 - **Acuerdos con proveedores**: comisiones por referidos con APIs como Duffel o LiteAPI en sus planes de producción. Aunque sería mejor tener APIs con datos reales de pago como la de Booking por nombrar alguna.
-

11. Plan de Producción y Recursos Humanos(RRHH)

El proyecto ha sido desarrollado íntegramente por un único desarrollador como Trabajo de Fin de Grado Superior de Desarrollo de Aplicaciones Web(CFGSDAW), asumiendo todos los roles del equipo:

Tabla 8. Roles que se han necesitado para crear BTT

Rol	Responsabilidad
Analista	Definición de requisitos y casos de uso
Diseñador	Diseño de interfaz y experiencia de usuario
Desarrollador frontend	Implementación en React con APIs externas
Desarrollador backend	Implementación en Spring Boot con JWT y JPA
DBA	Diseño y gestión de la base de datos MySQL
Tester	Pruebas manuales funcionales

Nota: Roles concretos para cada tarea realizada y sus funciones.

Planificación del proyecto:*Tabla 9. Tabla que muestra las fases, descripción y tiempo de cada fase*

Fase	Descripción	Duración estimada
Análisis	Requisitos, actores, casos de uso	1 semana
Diseño	Arquitectura, E-R, boceto de interfaces	1 semana
Implementación backend	Spring Boot, JPA, seguridad JWT	3 semanas
Implementación frontend	React, APIs externas, UI	4 semanas
Integración y pruebas	Conexión frontend-backend, pruebas manuales	1 semana
Documentación	Memoria del TFG	2 semanas
Total		12 semanas

Nota: Los datos no han sido medidos al milímetro y tampoco se han hecho en ese orden y numerosas veces se han tenido que corregir detalles hechas en fases anteriores en una fase posterior.

12. Plan de prevención de riesgos laborales

El desarrollo de software conlleva a su vez una serie de riesgos laborales asociados al trabajo prolongado frente a pantallas(luz azul) y en posición sedentaria. A continuación se identifican los principales riesgos y las medidas preventivas aplicables:

Tabla 10. Riesgos laborales y las medidas para prevenirlas

Riesgo	Medida preventiva
Fatiga visual por exposición prolongada a pantallas	Seguir la regla 20-20-20: cada 20 minutos mirar a 20 metros durante 20 segundos. Usar modo oscuro y ajustar el brillo de la pantalla.
Problemas musculoesqueléticos por mala postura	Mantener una postura ergonómica — espalda recta, pantalla a la altura de los ojos a un brazo de distancia, teclado y ratón a la altura del codo. Proporcionar asientos ergonómicos para que las vértebras descansen de forma idónea.
Estrés y fatiga mental	Realizar pausas regulares cada hora, establecer horarios de trabajo definidos y desconectar fuera del horario laboral.
Sedentarismo	Realizar actividad física regular fuera del horario de trabajo.
Síndrome del túnel carpiano (grave)	Usar reposamuñecas y realizar estiramientos de manos y muñecas periódicamente.
Dolor de cabeza por iluminación inadecuada	Asegurar una iluminación adecuada del espacio de trabajo, evitando reflejos en la pantalla. Acompañar de luz artificial tenue cálida si no disponemos de luz solar en el espacio de trabajo.

Nota: La profesión de programador junior se clasifica como una de riesgo bajo.

La normativa aplicable en España es la Ley 31/1995 de Prevención de Riesgos Laborales y el Real Decreto 488/1997 sobre disposiciones mínimas de seguridad y salud en el trabajo con equipos que incluyen pantallas de visualización.